



HOSPITAL DATABASE MANAGEMENT SYSTEM

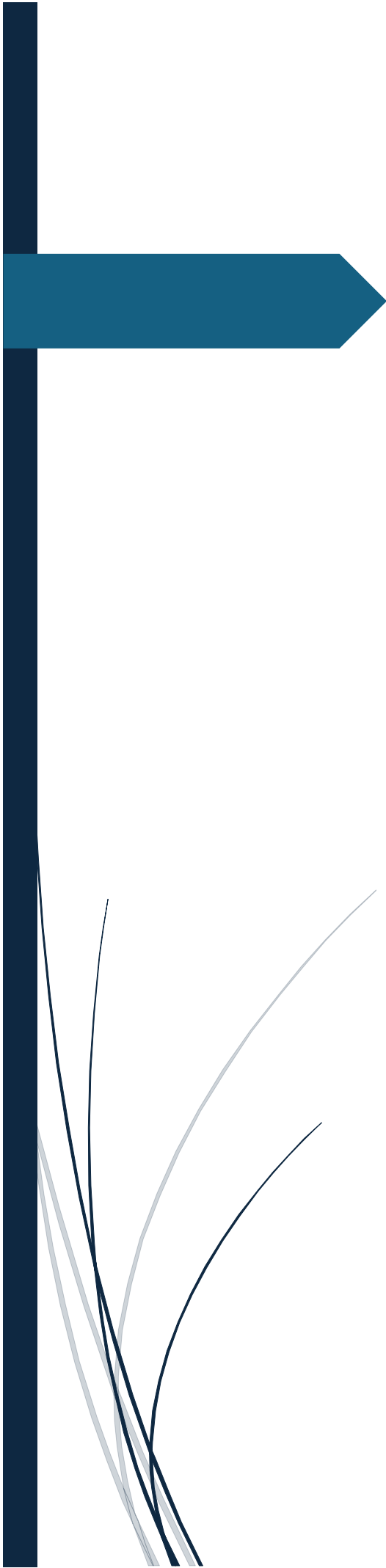
GROUP MEMBERS:

TEHREEM AHMAD (0025-BS-SP-CS-24)

WAFRA RAHMAN (0041-BS-SP-CS-24)

SUBMITTED TO:

SIR M.HAFEEZ



HOSPITAL MANAGEMENT SYSTEM (HMS) – DATABASE DESCRIPTION DOCUMENT

1. Introduction

The Hospital Management System (HMS) database is a fully normalized relational database designed to manage patient information, medical operations, administrative workflow, and financial activities within a hospital. The system ensures accuracy, integrity, and efficient retrieval of data across all departments.

2. Objective of the Database

The primary objectives of the HMS database are:

1. **Centralized Patient Management:** Maintain accurate, accessible patient records including demographics, history, insurance, encounters, prescriptions, and billing.
 2. **Operational Efficiency:** Support hospital workflows such as appointments, admissions, emergency cases, room allocation, and lab tests.
 3. **Data Integrity & Consistency:** Enforce reliable relationships using foreign keys, constraints, and normalization.
 4. **Financial Transparency:** Track billing, payments, insurance details, and medical charges.
 5. **Scalability:** Support future expansion, new departments, services, and digital transformation of hospital operations.
-

3. Scope of the Database

The HMS database covers:

Clinical Operations

- Patient demographics and health history
- Doctor profiles and department assignments
- Medical encounters (OPD/IPD/Emergency)
- Appointments & admissions
- Emergency visits
- Prescriptions, medicines, and lab tests
- Test reports

Administrative & Infrastructure Management

- Hospital departments
- Wards and room status

- Bed allocation

Financial Management

- Insurance details
- Billing operations
- Payment tracking

Exclusions (Out of Scope)

- Nursing schedules
 - External referrals
 - Pharmacy inventory
 - HR, payroll, staff attendance
-

4. System Architecture

The HMS follows a **three-tier architecture**:

1. Presentation Layer

- User interfaces (web/desktop/mobile apps)
- Views accessed by reception staff, doctors, admin, and finance department

2. Application Layer

- Business logic for:
 - Appointment scheduling
 - Admission workflows
 - Medical record processing
 - Billing & payment calculations

3. Database Layer (MySQL)

- Fully normalized relational tables
 - Enforced with **primary keys, foreign keys, unique constraints, actions on delete/update**, and suitable data types
 - Ensures data consistency, integrity, and secure storage
 - All queries, triggers, and stored procedures run in this layer
-

5. Entity Descriptions

Below is a precise description of each database entity:

1. Patient

Stores personal details including demographics, contact, city, email, and deceased status.

2. Patient_History

Maintains chronic diseases and medical notes linked to each patient.

3. Insurance

Records patient insurance providers and insurance status (active/expired).

4. Department

Defines medical departments (Cardiology, Neurology, etc.).

5. Doctor

Stores doctor profiles and assigns them to departments along with their consultation fee.

6. Ward

Represents hospital wards (Ward A, Ward B, etc.).

7. Room

Rooms within wards, including charges and availability/occupancy status.

8. Encounter

Represents any interaction of a patient with the hospital (OPD, IPD, Emergency).

9. Appointment

Scheduled time slots for OPD encounters.

10. Admission

Stores details of admitted patients: ward, room, admission/discharge times.

11. Emergency

Captures emergency visits and symptoms (for Emergency-type encounters).

12. Prescription

Medical prescriptions issued during encounters.

13. Medicine

List of medicines with associated cost.

14. Prescription_Medicine

Junction table linking prescriptions with medicines and dosage/duration.

15. Lab_Test

List of lab tests offered by hospital departments.

16. Test_Report

Test results for each encounter with date and result details.

17. Billing

Tracks billing for encounters including insurance involvement and total amount.

18. Payment

Records payment details for each bill including mode and timestamp.

6. ERD Summary with Cardinality & Modality

Key Relationship Rules (Cardinality + Modality)

Relationship	Cardinality	Modality	Explanation
Patient → Patient_History	1 : M	Mandatory on history side, optional on patient side	A patient may have multiple history records, but history cannot exist without patient.
Patient → Insurance	1 : M	Optional	A patient may have multiple insurance entries or none.
Department → Doctor	1 : M	Mandatory	Each doctor must belong to one department.
Ward → Room	1 : M	Mandatory	Each room belongs to one ward.
Patient → Encounter	1 : M	Mandatory	A patient can have many encounters.
Doctor → Encounter	1 : M	Mandatory	An encounter requires one doctor.
Encounter → Appointment	1 : 0..1	Optional	Only OPD encounters may have an appointment.
Encounter → Admission	1 : 0..1	Optional	Only IPD encounters have admissions.
Encounter → Emergency	1 : 0..1	Optional	Only emergency encounters have emergency records.
Encounter → Prescription	1 : M	Mandatory	An encounter may generate multiple prescriptions.
Prescription → Prescription_Medicine	1 : M	Mandatory	A prescription must have at least one medicine entry.
Medicine → Prescription_Medicine	1 : M	Mandatory	A medicine may appear in many prescriptions.

Relationship	Cardinality	Modality	Explanation
Department → Lab_Test	1 : M	Mandatory	A test belongs to one department.
Encounter → Test_Report	1 : M	Mandatory	A test report is linked to one encounter.
Lab_Test → Test_Report	1 : M	Mandatory	A test may have multiple reports.
Encounter → Billing	1 : M	Mandatory	Every encounter must be billed.
Billing → Payment	1 : M	Mandatory	A bill must have at least one payment entry.
Insurance → Billing	1 : M	Optional	Billing may or may not use insurance.

7. Importance of Good Database Design

A well-designed and normalized HMS database is essential because:

1. **Data Integrity:** Eliminates redundancy, prevents anomalies, and ensures consistent patient and billing data.
2. **Performance Optimization:** Efficient search, indexing, and query performance for high-volume hospital operations.
3. **Scalability:** Easily accommodates adding new departments, services, or features.
4. **Security:** Protects sensitive patient health information using constraints and controlled access.
5. **Error Reduction:** Reduces incorrect billing, duplicate patient records, and incomplete data dependencies.
6. **Improved Decision-Making:** Reliable data helps hospital administration analyze trends, patient load, and financial reports.
7. **Regulatory Compliance:** Helps meet healthcare data standards and audit requirements.

8. Conclusion

The Hospital Management System database is a comprehensive, normalized, and professionally structured system designed to support hospital operations across clinical, administrative, and financial domains. By ensuring strong relationships between entities, enforcing referential integrity, and maintaining scalable architecture, this database provides a foundation for efficient hospital management, improved patient care, and robust decision-making. The design allows for future expansion while maintaining stability, reliability, and high performance.

NOUNS (From All Entities + Attributes)

Patient Module

- patient
- name
- date
- birth
- gender
- contact
- city
- email
- history
- disease
- note

Insurance Module

- insurance
- provider
- policy
- coverage

Encounter Module

- encounter
- diagnosis
- visit
- symptoms

Billing Module

- bill
- amount
- status
- payment

Ward Module

- ward
- type

- capacity

Room Module

- room
- number
- availability

Doctor Module

- doctor
- specialization
- experience
- salary
- department

Department Module

- department
- name
- floor

Staff Module

- staff
- job
- phone
- salary

Appointment Module

- appointment
- date
- time
- status

Medicine Module

- medicine
- type
- category
- manufacturer
- price

Prescription Module

- prescription
- dosage
- frequency
- medication

Prescription_Medicine Module

- dosage
- timing

Lab Test Module

- test
- cost
- report
- result

Admission Module

- admission
- discharge
- summary
- recommendation

Payment Module

- payment method
- amount
- transaction

VERBS (Implicit Actions in the Entities)

(Even though database entities are nouns, many attributes imply an action.)

Patient / Encounter / Admission

- admit
- discharge
- diagnose
- treat
- record

- update

Appointments

- schedule
- cancel
- complete

Billing

- pay
- charge
- calculate

Prescription / Medicine

- prescribe
- take
- dispense
- administer

Lab Tests

- test
- conduct
- report
- analyze

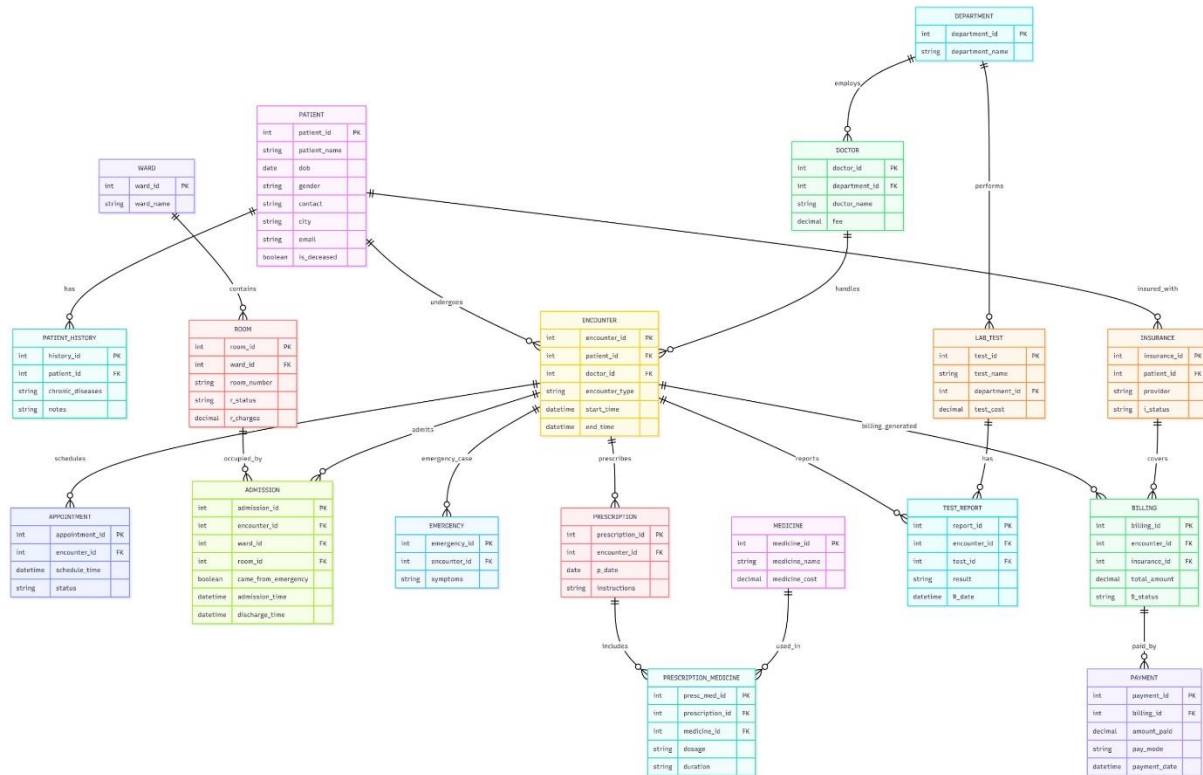
Staff / Doctor

- assist
- treat
- manage

System Actions

- insert
- update
- reference
- link

Relational Tables



DATABASE VALIDATION

Hospital Management System (HMS)

1. Validation of Entities

1. Patient Validations

1. Every patient must have a unique **patient_id**, ensuring reliable identification.
2. Mandatory fields such as name, gender, date of birth, and contact cannot be null.
3. The **dob** field must contain a valid date format.
4. Email must follow proper formatting, and if set UNIQUE, duplicates are not allowed.
5. Patients cannot be created without required demographic and contact information.

2. Patient History Validations

1. Each patient history entry must reference a valid **patient_id** through a foreign key.
2. **history_id** must be unique, ensuring no duplication of medical background records.
3. Chronic diseases and notes must contain meaningful text describing medical details.
4. A history record cannot exist for a non-existent patient.

3. Insurance Validations

1. Each insurance record must have a unique **insurance_id** for accurate linkage.
2. Insurance provider name must not be null and must contain valid text.
3. Policy details must follow proper data formats (e.g., numeric policy numbers).
4. Insurance entries can only be linked to patients through valid foreign keys.

4. Encounter Validations

1. Each encounter must reference a valid **patient_id**, ensuring encounters cannot exist independently.
2. **encounter_date** must be a valid date and cannot be null.
3. Diagnosis and encounter notes must contain meaningful information.
4. Encounter_ID must be unique to distinguish separate hospital visits.

5. Billing Validations

1. Each billing record must reference a valid **encounter_id**.
2. **insurance_id**, when present, must be valid; otherwise, NULL is allowed if the patient is uninsured.
3. Total amount must be a numeric, non-negative value.
4. Billing status must contain defined values (e.g., 'paid', 'unpaid').
5. Billing entries cannot exist without a valid encounter.

6. Ward Validations

1. Every ward must have a unique **ward_id** for internal tracking.
2. Ward name and ward type must be provided and cannot be null.
3. Capacity must be numeric and must represent the maximum number of patients logically.
4. Wards must follow correct category classifications (e.g., ICU, General, Private).

7. Room Validations

1. Each room must belong to a valid **ward_id**, protected by a foreign key.
2. Room number must be unique within each ward.

3. Room type must be provided (e.g., single, shared).
 4. Availability status must follow defined values.
 5. Rooms cannot exist without being assigned to a ward.
-

8. Doctor Validations

1. Each doctor must have a unique **doctor_id** for identification.
 2. Doctor name, specialization, and contact information cannot be empty.
 3. Annual salary and experience must be numeric and valid.
 4. Doctors must belong to valid departments via **dept_id**.
-

9. Department Validations

1. Each department requires a unique **dept_id**.
 2. Department name must be non-empty and meaningful.
 3. Optional information must follow appropriate formatting.
-

10. Appointment Validations

1. Each appointment must reference a valid **patient_id** and **doctor_id**.
 2. Appointment date and time must follow valid formats.
 3. Appointment status must follow logical values (scheduled, completed, cancelled).
 4. No appointment record may exist without meeting all mandatory fields.
-

11. Medicine Validations

1. Medicine must have a unique **medicine_id**.
 2. Name, type, and manufacturer must not be null.
 3. Price must be numeric and non-negative.
 4. Medicines must belong to a valid category through **category_id**.
-

12 Medicine Category Validations

1. Category_ID must be unique.
2. Category name cannot be null.
3. Records must use proper text formats.

13. Prescription Validations

1. Each prescription must be linked to a valid **appointment_id**.
2. Prescription date must be a valid date and cannot be null.
3. Notes must contain meaningful clinical information.
4. No prescription can exist without a valid appointment.

14. Prescription_Medicine Validations (Many-to-Many Module)

1. Every record must reference a valid **presc_id** and **medicine_id**.
2. Duplicate entries for the same prescription-medicine pair are prevented by a composite key.
3. Dosage and frequency must contain valid medical details.
4. Invalid associations are blocked by foreign key constraints.

15. Laboratory Test Validations

1. Each test must have a unique **labtest_id**.
2. Test name and cost must be non-null, with cost being numeric.
3. Departments conducting the test must be valid.

16. Lab Test Report Validations

1. Each report must be associated with a valid **labtest_id** and **patient_id**.
2. Report date must be valid.
3. Results must contain correctly formatted diagnostic text.
4. **report_id** must be unique.

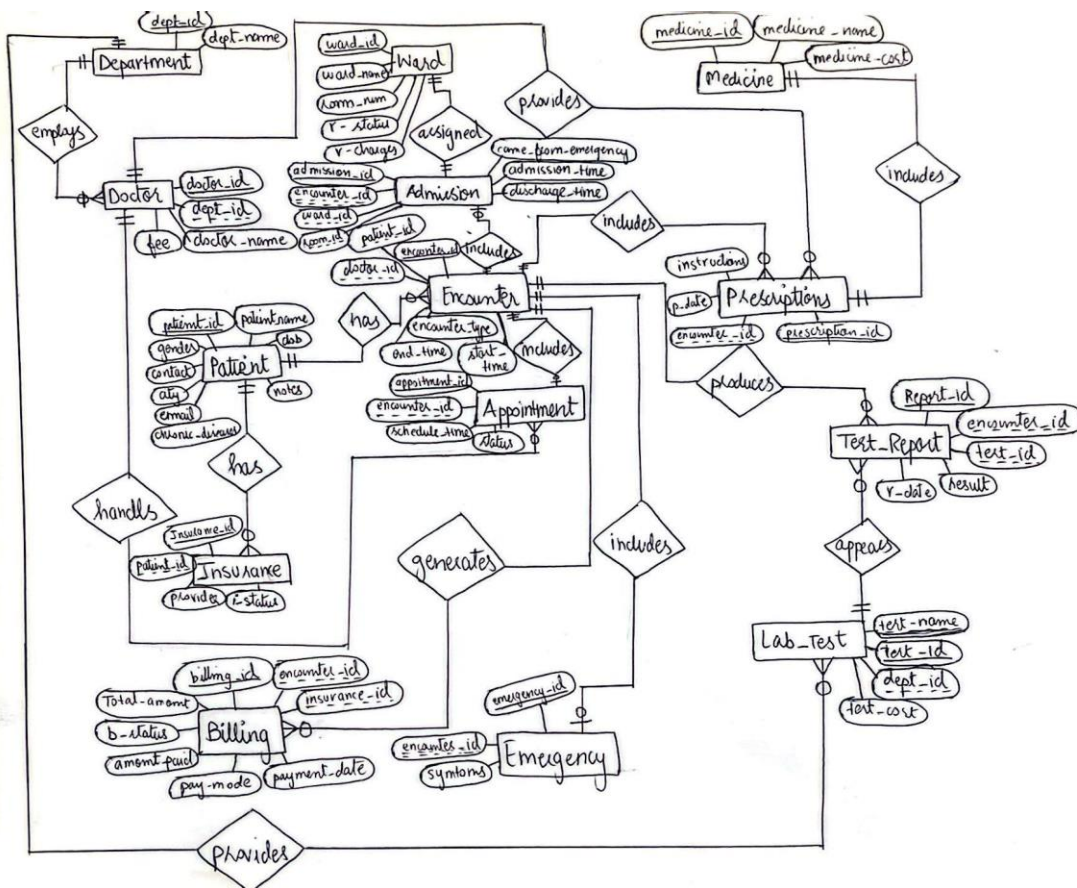
17. Admission Validations

1. Every admission must reference a valid **patient_id**, **room_id**, and **doctor_id**.
 2. Admission and discharge dates must be valid; discharge date cannot be before admission date.
 3. Admission status must follow defined values.
 4. **admission_id** must be unique.
-

18. Payment Validations

1. Each payment must reference a valid **billing_id**.
2. Amount paid must be numeric and non-negative.
3. Payment method must contain valid text.
4. Payment status must follow defined values.
5. **payment_id** must be unique for transaction tracking.

Unresolved ERD



Normalization Summary

The unnormalized ERD contained embedded and repeating data within larger entities, causing redundancy, update anomalies, and maintenance challenges. Normalization separated these into dedicated entities to ensure atomicity, integrity, and scalability:

Patient_History: Moved from Patient to record multiple medical histories per patient.

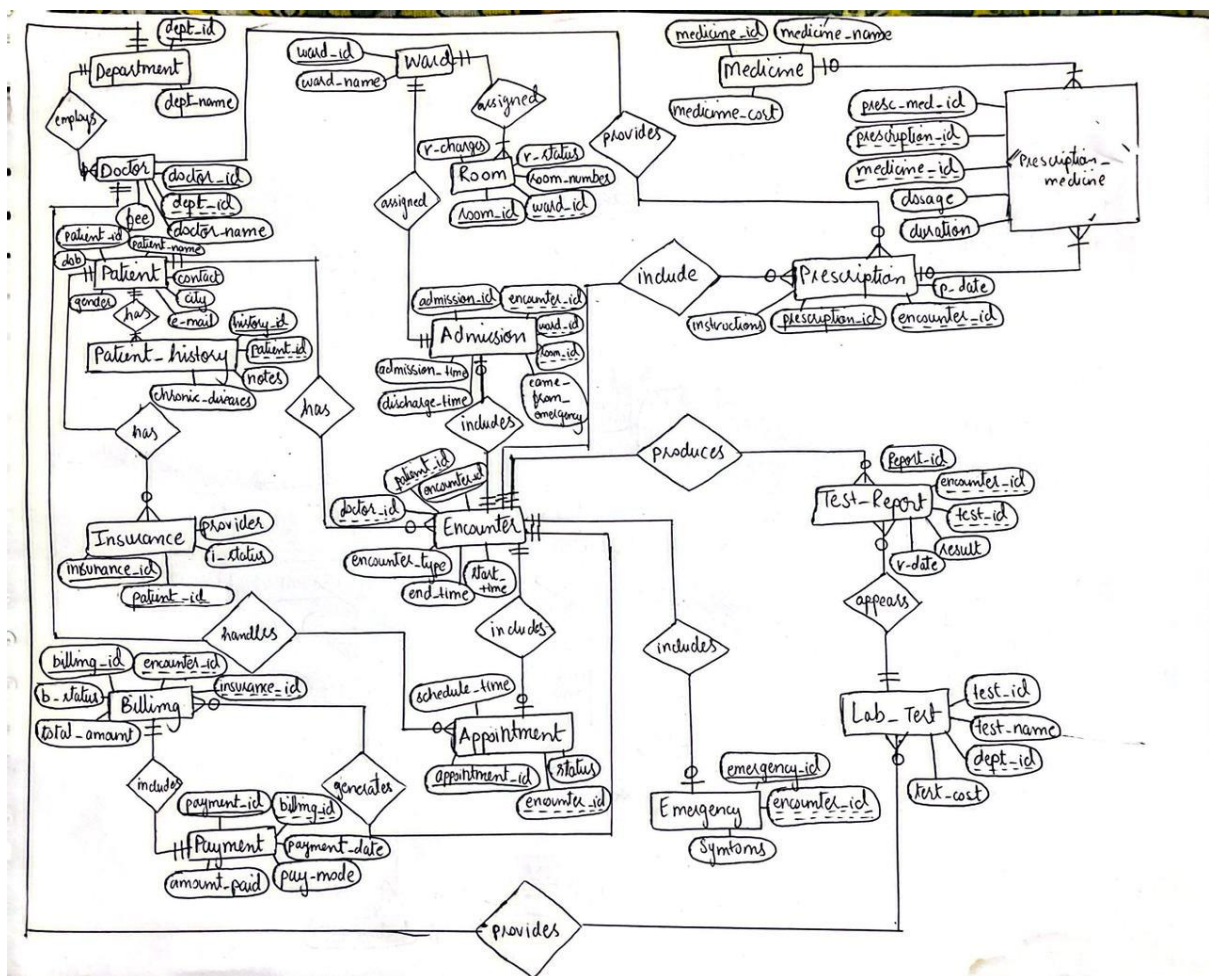
Room: Separated from Ward/Admission to track multiple rooms, charges, and availability independently.

Prescription_Medicine: Introduced as a junction table to handle many-to-many relationships between prescriptions and medicines, storing dosage and duration.

Lab_Test & Test_Report: Split from Encounter to track multiple tests per encounter with individual results.

Billing & Payment: Normalized to track total charges and multiple transactions per encounter, with optional Insurance linkage.

NORMALIZED ERD



SQL CODE

```
-- *****

-- HOSPITAL MANAGEMENT SYSTEM (NORMALIZED ERD)

-- MySQL SQL Workbench DDL

-- *****


CREATE DATABASE IF NOT EXISTS hosp_dbms;

USE hosp_dbms;


-- =====

-- 1. PATIENT

-- =====

CREATE TABLE Patient (

  patient_id    INT AUTO_INCREMENT PRIMARY KEY,

  patient_name  VARCHAR(100) NOT NULL,

  dob          DATE,

  gender        ENUM('Male','Female','Other'),

  contact       VARCHAR(20),

  city          VARCHAR(255),

  email         VARCHAR(100),

  is_deceased   boolean DEFAULT FALSE

) ENGINE=InnoDB;


-- =====

-- 2. PATIENT HISTORY

-- =====

CREATE TABLE Patient_History (

  history_id    INT AUTO_INCREMENT PRIMARY KEY,

  patient_id    INT NOT NULL,

  chronic_diseases VARCHAR(255),

  notes         TEXT,


  FOREIGN KEY (patient_id)

  REFERENCES Patient(patient_id)

  ON DELETE CASCADE
```

```

        ON UPDATE CASCADE
    ) ENGINE=InnoDB;

-- =====

-- 3. INSURANCE

-- =====

CREATE TABLE Insurance (
    insurance_id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    provider VARCHAR(100) NOT NULL,
    i_status ENUM('active','expired')DEFAULT 'active',

    CONSTRAINT fk_insurance_patient
        FOREIGN KEY (patient_id)
        REFERENCES Patient(patient_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
    ) ENGINE=InnoDB;

-- =====

-- 4. DEPARTMENT

-- =====

CREATE TABLE Department (
    department_id INT AUTO_INCREMENT PRIMARY KEY,
    department_name VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

-- =====

-- 5. DOCTOR

-- =====

CREATE TABLE Doctor (
    doctor_id INT AUTO_INCREMENT PRIMARY KEY,
    department_id INT NOT NULL,
    doctor_name VARCHAR(100) NOT NULL,
    fee decimal(10,2) NOT NULL,

    CONSTRAINT fk_doctor_department
        FOREIGN KEY (department_id)

```

```

REFERENCES Department(department_id)

ON DELETE RESTRICT

ON UPDATE CASCADE
) ENGINE=InnoDB;

-- =====

-- 6. WARD

-- =====

CREATE TABLE Ward (

ward_id INT AUTO_INCREMENT PRIMARY KEY,

ward_name VARCHAR(100)

) ENGINE=InnoDB;

-- =====

-- 7. ROOM

-- =====

CREATE TABLE Room (

room_id INT AUTO_INCREMENT PRIMARY KEY,

ward_id INT NOT NULL,

room_number VARCHAR(20),

r_status ENUM('A','O','M') DEFAULT 'A',

r_charges decimal(10,2) NOT NULL,


CONSTRAINT fk_room_ward

FOREIGN KEY (ward_id)

REFERENCES Ward(ward_id)

ON DELETE RESTRICT

ON UPDATE CASCADE

) ENGINE=InnoDB;

-- =====

-- 8. ENCOUNTER

-- =====

CREATE TABLE Encounter (

encounter_id INT AUTO_INCREMENT PRIMARY KEY,

patient_id INT NOT NULL,

doctor_id INT NOT NULL,

encounter_type ENUM('OPD','IPD','EMERGENCY') NOT NULL,

```

```

start_time DATETIME,

end_time DATETIME,

CONSTRAINT fk_encounter_patient

    FOREIGN KEY (patient_id) REFERENCES Patient(patient_id)

    ON DELETE CASCADE

    ON UPDATE CASCADE,

CONSTRAINT fk_encounter_doctor

    FOREIGN KEY (doctor_id) REFERENCES Doctor(doctor_id)

    ON DELETE RESTRICT

    ON UPDATE CASCADE

) ENGINE=InnoDB;

-- =====

-- 9. APPOINTMENT

-- (0..1 per encounter)

-- =====

CREATE TABLE Appointment (

    appointment_id INT AUTO_INCREMENT PRIMARY KEY,

    encounter_id INT NOT NULL,

    schedule_time DATETIME NOT NULL,

    status ENUM('Scheduled','Completed','Cancelled') DEFAULT 'Scheduled',

    CONSTRAINT fk_appointment_encounter

        FOREIGN KEY (encounter_id) REFERENCES Encounter(encounter_id)

        ON DELETE CASCADE

        ON UPDATE CASCADE

) ENGINE=InnoDB;

-- =====

-- 10. ADMISSION

-- (0..1 per encounter)

-- =====

CREATE TABLE Admission (

    admission_id INT AUTO_INCREMENT PRIMARY KEY,

    encounter_id INT UNIQUE,

    ward_id INT NOT NULL,

    room_id INT NOT NULL,

    came_from_emergency BOOLEAN DEFAULT FALSE,

    admission_time DATETIME,

```

discharge_time DATETIME,

CONSTRAINT fk_admission_encounter

FOREIGN KEY (encounter_id)

REFERENCES Encounter(encounter_id)

ON DELETE CASCADE

ON UPDATE CASCADE,

CONSTRAINT fk_admission_ward

FOREIGN KEY (ward_id)

REFERENCES Ward(ward_id)

ON DELETE RESTRICT

ON UPDATE CASCADE,

CONSTRAINT fk_admission_room

FOREIGN KEY (room_id)

REFERENCES Room(room_id)

ON DELETE RESTRICT

ON UPDATE CASCADE

) ENGINE=InnoDB;

-- =====

-- 11. EMERGENCY

-- (0..1 per encounter)

-- =====

CREATE TABLE Emergency (

emergency_id INT AUTO_INCREMENT PRIMARY KEY,

encounter_id INT UNIQUE,

symptoms VARCHAR(100),

CONSTRAINT fk_emergency_encounter

FOREIGN KEY (encounter_id)

REFERENCES Encounter(encounter_id)

ON DELETE CASCADE

ON UPDATE CASCADE

) ENGINE=InnoDB;

-- =====

-- 12. PRESCRIPTION

```

-- =====

CREATE TABLE Prescription (

    prescription_id INT AUTO_INCREMENT PRIMARY KEY,

    encounter_id INT NOT NULL,

    p_date DATE,

    instructions TEXT,


    CONSTRAINT fk_prescription_encounter

        FOREIGN KEY (encounter_id)

            REFERENCES Encounter(encounter_id)

            ON DELETE CASCADE

            ON UPDATE CASCADE


) ENGINE=InnoDB;


-- =====

-- 13. MEDICINE

-- =====

CREATE TABLE Medicine (

    medicine_id INT AUTO_INCREMENT PRIMARY KEY,

    medicine_name VARCHAR(100) NOT NULL,

    medicine_cost DECIMAL(10,2) NOT NULL

) ENGINE=InnoDB;


-- =====

-- 14. PRESCRIPTION_MEDICINE (junction table)

-- =====

CREATE TABLE Prescription_Medicine (

    presc_med_id INT AUTO_INCREMENT PRIMARY KEY,

    prescription_id INT NOT NULL,

    medicine_id INT NOT NULL,

    dosage VARCHAR(100),

    duration VARCHAR(100),


    CONSTRAINT fk_pm_prescription

        FOREIGN KEY (prescription_id)

            REFERENCES Prescription(prescription_id)

            ON DELETE CASCADE

```

```

        ON UPDATE CASCADE,

CONSTRAINT fk_pm_medicine

    FOREIGN KEY (medicine_id)

    REFERENCES Medicine(medicine_id)

    ON DELETE RESTRICT

    ON UPDATE CASCADE
) ENGINE=InnoDB;

-- =====

-- 15. LAB TEST

-- =====

CREATE TABLE Lab_Test (

    test_id    INT AUTO_INCREMENT PRIMARY KEY,

    test_name  VARCHAR(100),

    department_id INT NOT NULL,

    test_cost  DECIMAL(10,2) NOT NULL,

    CONSTRAINT fk_labtest_department

        FOREIGN KEY (department_id)

        REFERENCES Department(department_id)

        ON DELETE RESTRICT

        ON UPDATE CASCADE
) ENGINE=InnoDB;

-- =====

-- 16. TEST REPORT

-- =====

CREATE TABLE Test_Report (

    report_id  INT AUTO_INCREMENT PRIMARY KEY,

    encounter_id INT NOT NULL,

    test_id    INT NOT NULL,

    result     VARCHAR(100),

    R_date     DATETIME NOT NULL,

    CONSTRAINT fk_report_encounter

        FOREIGN KEY (encounter_id)

        REFERENCES Encounter(encounter_id)

```

```

        ON DELETE CASCADE

        ON UPDATE CASCADE,

CONSTRAINT fk_report_test

    FOREIGN KEY (test_id)

    REFERENCES Lab_Test(test_id)

    ON DELETE RESTRICT

    ON UPDATE CASCADE

) ENGINE=InnoDB;

-- =====

-- 17. BILLING

-- =====

CREATE TABLE Billing (

    billing_id  INT AUTO_INCREMENT PRIMARY KEY,

    encounter_id INT NOT NULL,

    insurance_id INT,

    total_amount DECIMAL(10,2),

    B_status    ENUM('paid','unpaid') DEFAULT 'unpaid',

CONSTRAINT fk_billing_encounter

    FOREIGN KEY (encounter_id)

    REFERENCES Encounter(encounter_id)

    ON DELETE CASCADE

    ON UPDATE CASCADE,

        CONSTRAINT fk_billing_insurance

    FOREIGN KEY (insurance_id)

    REFERENCES insurance(insurance_id)

    ON DELETE SET NULL

    ON UPDATE CASCADE

) ENGINE=InnoDB;

-- =====

-- 18. PAYMENT

-- =====

CREATE TABLE Payment (

    payment_id  INT AUTO_INCREMENT PRIMARY KEY,

    billing_id  INT NOT NULL,

```



```

amount_paid  DECIMAL(10,2),

pay_mode     ENUM('cash','card','insurance') DEFAULT 'cash',

payment_date DATETIME NOT NULL,


CONSTRAINT fk_payment_billing

FOREIGN KEY (billing_id)

REFERENCES Billing(billing_id)

ON DELETE CASCADE

ON UPDATE CASCADE

) ENGINE=InnoDB;


-- ===== 1. Patients (30) =====

INSERT INTO Patient (patient_name, dob, gender, contact, city, email, is_deceased) VALUES

('Ali Khan', '1980-05-12', 'Male', '1234567890', 'Lahore', 'ali@example.com', false),

('Alia Baloch', '1992-08-22', 'Female', '2345678901', 'Lahore', 'alia@example.com', false),

('Sanam Ahmar', '1975-03-30', 'Female', '3456789012', 'Karachi', 'sanam@example.com', false),

('Babar Hassan', '1985-11-15', 'Male', '4567890123', 'Islamabad', 'babar@example.com', false),

('Fatima Ahmad', '1990-01-25', 'Female', '5678901234', 'Rawalpindi', 'fatimaahmad@example.com', false),

('Hassan Rafiq', '1982-06-10', 'Male', '6789012345', 'Peshawar', 'hassan@example.com', false),

('Nadia Qureshi', '1995-12-05', 'Female', '7890123456', 'Multan', 'nadia@example.com', false),

('Imran Malik', '1978-09-21', 'Male', '8901234567', 'Faisalabad', 'imran@example.com', false),

('Sara Javed', '1988-03-15', 'Female', '9012345678', 'Quetta', 'sara@example.com', false),

('Zain Abbas', '1993-07-30', 'Male', '9123456789', 'Hyderabad', 'zain@example.com', false),

('Ayesha Tariq', '1984-04-12', 'Female', '9234567890', 'Lahore', 'ayesha@example.com', false),

('Omar Saeed', '1991-08-08', 'Male', '9345678901', 'Karachi', 'omar@example.com', false),

('Maryam Khan', '1987-12-20', 'Female', '9456789012', 'Islamabad', 'maryam@example.com', false),

('Ahmed Raza', '1983-07-05', 'Male', '9567890123', 'Lahore', 'ahmed@example.com', false),

('Sana Iqbal', '1990-02-17', 'Female', '9678901234', 'Rawalpindi', 'sana@example.com', false),

('Bilal Jameel', '1979-11-01', 'Male', '9789012345', 'Peshawar', 'bilal@example.com', false),

('Hina Farooq', '1992-06-10', 'Female', '9890123456', 'Multan', 'hina@example.com', false),

('Waseem Shah', '1985-03-22', 'Male', '9901234567', 'Faisalabad', 'waseem@example.com', false),

('Saba Noreen', '1988-09-15', 'Female', '9012345670', 'Quetta', 'saba@example.com', false),

```

('Raza Ali', '1993-12-25', 'Male', '9123456701', 'Hyderabad', 'raza@example.com', false),
 ('Mariam Aslam', '1986-10-18', 'Female', '9234567012', 'Lahore', 'mariam@example.com', false),
 ('Fahad Khan', '1981-05-30', 'Male', '9345670123', 'Karachi', 'fahad@example.com', false),
 ('Iqra Siddiqui', '1994-01-10', 'Female', '9456701234', 'Islamabad', 'iqra@example.com', false),
 ('Shahbaz Rafiq', '1977-08-14', 'Male', '9567012345', 'Lahore', 'shahbaz@example.com', false),
 ('Sonia Malik', '1989-03-02', 'Female', '9670123456', 'Rawalpindi', 'sonia@example.com', false),
 ('Tariq Javed', '1982-11-20', 'Male', '9781234567', 'Peshawar', 'tariq@example.com', false),
 ('Amna Bukhari', '1995-07-25', 'Female', '9892345678', 'Multan', 'amna@example.com', false),
 ('Hamza Ali', '1980-09-30', 'Male', '9903456789', 'Faisalabad', 'hamza@example.com', false),
 ('Neha Shah', '1991-04-12', 'Female', '9014567890', 'Quetta', 'neha@example.com', false),
 ('Adil Raza', '1985-06-18', 'Male', '9125678901', 'Hyderabad', 'adil@example.com', false);

-- ===== 2. Patient History =====

INSERT INTO Patient_History (patient_id, chronic_diseases, notes) VALUES

(1, 'Diabetes', 'Requires regular monitoring'),
 (2, 'Hypertension', 'On medication'),
 (3, 'Asthma', 'Carry inhaler'),
 (4, 'None', 'Healthy'),
 (5, 'Thyroid', 'Needs monthly checkup'),
 (6, 'Heart disease', 'Monitor BP and cholesterol'),
 (7, 'Allergy', 'Avoid pollen and dust'),
 (8, 'None', 'Healthy'),
 (9, 'Diabetes', 'Diet controlled'),
 (10, 'Hypertension', 'Occasional headaches'),
 (11, 'Asthma', 'Carry inhaler'),
 (12, 'Diabetes', 'Regular insulin'),
 (13, 'None', 'Healthy'),
 (14, 'Hypertension', 'On meds'),
 (15, 'Thyroid', 'Monitor TSH'),
 (16, 'Heart disease', 'Follow cardiologist'),
 (17, 'Allergy', 'Seasonal allergies'),
 (18, 'None', 'Healthy'),
 (19, 'Diabetes', 'Monitor glucose'),
 (20, 'Hypertension', 'BP check monthly'),
 (21, 'Thyroid', 'Medication daily'),
 (22, 'None', 'Healthy'),
 (23, 'Asthma', 'Inhaler needed'),
 (24, 'Heart disease', 'ECG monthly'),

```
(25,'Allergy','Avoid dust'),
(26,'Diabetes','Diet controlled'),
(27,'Hypertension','Medication regular'),
(28,'None','Healthy'),
(29,'Thyroid','Checkup every 6 months'),
(30,'Heart disease','Follow doctor instructions');
```

```
-- ===== 3. Insurance =====
```

```
INSERT INTO Insurance (patient_id, provider, i_status) VALUES
```

```
(1, 'BlueCross', 'active'),
(2, 'Aetna', 'active'),
(3, 'Cigna', 'expired'),
(6, 'State Life', 'active'),
(7, 'Jubilee', 'active'),
(12,'Medicare','active'),
(15,'Aetna','expired'),
(20,'BlueCross','active'),
(22,'State Life','active'),
(25,'Jubilee','active');
```

```
-- ===== 4. Department =====
```

```
INSERT INTO Department (department_name) VALUES
```

```
('Cardiology'),('Neurology'),('Orthopedics'),('Radiology'),('Pathology'),('General Surgery');
```

```
-- ===== 5. Doctors =====
```

```
INSERT INTO Doctor (department_id, doctor_name, fee) VALUES
```

```
(1, 'Dr. Fatima Ali', 500.00),
(2, 'Dr. Zohaib', 600.00),
(3, 'Dr. Shazia', 400.00),
(4, 'Dr. Amina', 300.00),
(5, 'Dr. M. Mohib', 200.00),
(6, 'Dr. Tariq', 450.00);
```

```
-- ===== 6. Wards =====
```

```
INSERT INTO Ward (ward_name) VALUES
```

```
('Ward A'),('Ward B'),('Ward C');
```

```
-- ===== 7. Rooms =====
```

```
INSERT INTO Room (ward_id, room_number, r_status, r_charges) VALUES
```

```
(1, '101', 'A', 1000.00),  
(1, '102', 'O', 1200.00),  
(2, '201', 'A', 1500.00),  
(2, '202', 'M', 1800.00),  
(3, '301', 'A', 2000.00),  
(3, '302', 'A', 2200.00);
```

```
-- ===== 8. Encounters (30) =====
```

```
INSERT INTO Encounter (patient_id, doctor_id, encounter_type, start_time, end_time) VALUES
```

```
(1, 1, 'OPD', '2025-11-01 09:00:00', '2025-11-01 09:30:00'),  
(2, 2, 'IPD', '2025-11-02 10:00:00', '2025-11-10 15:00:00'),  
(3, 3, 'EMERGENCY', '2025-11-03 22:00:00', '2025-11-04 06:00:00'),  
(4, 1, 'OPD', '2025-11-04 11:00:00', '2025-11-04 11:20:00'),  
(5, 4, 'IPD', '2025-11-05 12:00:00', '2025-11-12 14:00:00'),  
(6, 6, 'OPD', '2025-11-06 09:15:00', '2025-11-06 09:50:00'),  
(7, 2, 'EMERGENCY', '2025-11-07 23:00:00', '2025-11-08 06:00:00'),  
(8, 3, 'IPD', '2025-11-08 14:00:00', '2025-11-15 16:00:00'),  
(9, 1, 'OPD', '2025-11-09 10:30:00', '2025-11-09 11:00:00'),  
(10, 5, 'IPD', '2025-11-10 12:00:00', '2025-11-18 13:00:00'),  
(11, 4, 'OPD', '2025-10-01 11:00:00', '2025-10-01 11:30:00'),  
(12, 3, 'OPD', '2025-09-15 09:00:00', '2025-09-15 09:30:00'),  
(13, 2, 'EMERGENCY', '2025-08-20 22:00:00', '2025-08-21 06:00:00'),  
(14, 5, 'IPD', '2025-07-10 10:00:00', '2025-07-17 12:00:00'),  
(15, 1, 'OPD', '2025-06-05 12:00:00', '2025-06-05 12:30:00'),  
(16, 6, 'OPD', '2025-05-22 09:15:00', '2025-05-22 09:50:00'),  
(17, 2, 'EMERGENCY', '2025-04-18 23:00:00', '2025-04-19 06:00:00'),  
(18, 3, 'IPD', '2025-03-12 14:00:00', '2025-03-19 16:00:00'),  
(19, 1, 'OPD', '2025-02-09 10:30:00', '2025-02-09 11:00:00'),  
(20, 5, 'IPD', '2025-01-10 12:00:00', '2025-01-18 13:00:00'),  
(21, 4, 'OPD', '2024-12-06 10:00:00', '2024-12-06 10:30:00'),  
(22, 3, 'OPD', '2024-11-22 09:00:00', '2024-11-22 09:30:00'),  
(23, 2, 'EMERGENCY', '2024-10-17 22:00:00', '2024-10-18 06:00:00'),  
(24, 5, 'IPD', '2024-09-04 10:00:00', '2024-09-11 12:00:00'),  
(25, 1, 'OPD', '2024-08-05 12:00:00', '2024-08-05 12:30:00'),  
(26, 6, 'EMERGENCY', '2024-07-06 23:00:00', '2024-07-07 06:00:00'),  
(27, 2, 'OPD', '2024-06-09 09:00:00', '2024-06-09 09:30:00'),  
(28, 3, 'IPD', '2024-05-10 14:00:00', '2024-05-18 16:00:00'),
```

```
(29, 1, 'OPD', '2024-04-12 10:30:00', '2024-04-12 11:00:00'),  
(30, 5, 'EMERGENCY', '2024-03-16 22:00:00', '2024-03-17 06:00:00');  
  
-- ===== 9. Appointments (approx. 20) =====
```

```
INSERT INTO Appointment (encounter_id, schedule_time, status) VALUES
```

```
(1,'2025-11-01 09:00:00','Completed'),  
(2,'2025-11-02 10:00:00','Completed'),  
(4,'2025-11-04 11:00:00','Completed'),  
(6,'2025-11-06 09:15:00','Completed'),  
(9,'2025-11-09 10:30:00','Completed'),  
(11,'2025-10-01 11:00:00','Completed'),  
(12,'2025-09-15 09:00:00','Completed'),  
(14,'2025-07-10 10:00:00','Completed'),  
(15,'2025-06-05 12:00:00','Completed'),  
(16,'2025-05-22 09:15:00','Completed'),  
(19,'2025-02-09 10:30:00','Completed'),  
(21,'2024-12-06 10:00:00','Completed'),  
(22,'2024-11-22 09:00:00','Completed'),  
(24,'2024-09-04 10:00:00','Completed'),  
(25,'2024-08-05 12:00:00','Completed'),  
(27,'2024-06-09 09:00:00','Completed'),  
(29,'2024-04-12 10:30:00','Completed');
```

```
-- ===== 10. Admissions (approx. 15) =====
```

```
INSERT INTO Admission (encounter_id, ward_id, room_id, came_from_emergency, admission_time, discharge_time) VALUES
```

```
(2,1,1,FALSE,'2025-11-02 10:30:00','2025-11-10 15:00:00'),  
(5,2,3,TRUE,'2025-11-05 12:30:00','2025-11-12 14:00:00'),  
(8,3,5,FALSE,'2025-11-08 14:30:00','2025-11-15 16:00:00'),  
(10,3,6,TRUE,'2025-11-10 12:30:00','2025-11-18 13:00:00'),  
(11,1,2,FALSE,'2025-10-01 11:30:00','2025-10-05 12:00:00'),  
(13,2,3,TRUE,'2025-08-20 22:30:00','2025-08-21 06:00:00'),  
(15,3,5,FALSE,'2025-06-05 12:30:00','2025-06-12 14:00:00'),  
(18,1,1,FALSE,'2025-03-12 14:30:00','2025-03-19 16:00:00'),  
(20,2,3,TRUE,'2025-01-10 12:30:00','2025-01-18 13:00:00'),  
(22,3,5,FALSE,'2024-11-22 10:30:00','2024-11-28 15:00:00'),  
(25,1,2,TRUE,'2024-08-05 12:30:00','2024-08-12 14:00:00'),  
(28,2,3,FALSE,'2024-05-10 12:30:00','2024-05-18 13:00:00');
```

```
-- ===== 11. Emergency (approx. 10) =====
```

INSERT INTO Emergency (encounter_id, symptoms) VALUES

(3,'Severe headache and dizziness'),
(7,'Acute chest pain'),
(13,'High fever and dehydration'),
(17,'Chest tightness and cough'),
(26,'Severe abdominal pain'),
(30,'Trauma and bleeding'),
(23,'Sudden dizziness and nausea');

-- ===== 12. Prescriptions (approx. 25) =====

INSERT INTO Prescription (encounter_id, p_date, instructions) VALUES

(1,'2025-11-01','Take medicines after meals'),
(2,'2025-11-02','Rest and monitor blood pressure'),
(3,'2025-11-03','Immediate hydration and observation'),
(4,'2025-11-04','Pain relief as needed'),
(6,'2025-11-06','Take medicine before breakfast'),
(9,'2025-11-09','Check blood sugar daily'),
(11,'2025-10-01','Painkillers as required'),
(12,'2025-09-15','Monitor glucose'),
(13,'2025-08-20','IV fluids and antibiotics'),
(14,'2025-07-10','Medication as prescribed'),
(15,'2025-06-05','Take tablets twice daily'),
(16,'2025-05-22','Rest and hydration'),
(17,'2025-04-18','Monitor vitals regularly'),
(18,'2025-03-12','Diet control and medication'),
(19,'2025-02-09','Regular checkup'),
(20,'2025-01-10','Blood pressure monitoring'),
(21,'2024-12-06','Vitamin supplements'),
(22,'2024-11-22','Routine medications'),
(23,'2024-10-17','Emergency care instructions'),
(24,'2024-09-04','Medication as needed'),
(25,'2024-08-05','Pain relief and rest'),
(26,'2024-07-06','IV hydration'),
(27,'2024-06-09','Daily medication'),
(28,'2024-05-10','Follow-up tests'),
(29,'2024-04-12','Routine prescription'),
(30,'2024-03-16','Emergency treatment');

-- ===== 13. Medicines =====

INSERT INTO Medicine (medicine_name, medicine_cost) VALUES

('Paracetamol', 50.00),
('Amoxicillin', 100.00),
('Ibuprofen', 80.00),
('Metformin', 120.00),
('Omeprazole', 90.00),
('Aspirin', 60.00),
('Cough Syrup', 70.00);

-- ===== 14. Prescription_Medicine =====

INSERT INTO Prescription_Medicine (prescription_id, medicine_id, dosage, duration) VALUES

(1,1,'1 tablet twice daily','5 days'),
(1,5,'1 capsule daily','7 days'),
(2,4,'500 mg twice daily','10 days'),
(2,2,'1 capsule daily','7 days'),
(3,3,'1 tablet thrice daily','3 days'),
(4,6,'1 tablet daily','5 days'),
(6,7,'10 ml twice daily','7 days'),
(5,4,'500 mg once daily','10 days'),
(11,1,'1 tablet twice daily','5 days'),
(12,2,'1 capsule daily','7 days'),
(13,3,'1 tablet thrice daily','3 days'),
(14,4,'500 mg twice daily','7 days'),
(15,5,'1 capsule daily','5 days'),
(16,6,'1 tablet daily','7 days'),
(17,7,'10 ml twice daily','7 days'),
(18,1,'1 tablet twice daily','5 days'),
(19,2,'1 capsule daily','7 days'),
(20,3,'1 tablet thrice daily','5 days'),
(21,4,'500 mg twice daily','5 days'),
(22,5,'1 capsule daily','7 days'),
(23,6,'1 tablet daily','7 days'),
(24,7,'10 ml twice daily','7 days'),
(25,1,'1 tablet twice daily','5 days');

-- ===== 15. Lab Tests =====

INSERT INTO Lab_Test (test_name, department_id, test_cost) VALUES

```
('CBC',5,200.00),
('X-Ray',4,500.00),
('MRI',2,1500.00),
('ECG',1,300.00),
('Ultrasound',4,700.00)
;
```

```
-- ===== 16. Test Reports =====
```

```
INSERT INTO Test_Report (encounter_id, test_id, result, R_date) VALUES
```

```
(1,4,'Normal','2025-11-01 10:00:00'),
(2,1,'Slight anemia','2025-11-03 08:00:00'),
(3,3,'No abnormalities','2025-11-04 05:00:00'),
(6,5,'Gallstones detected','2025-11-06 10:00:00'),
(9,2,'Fracture ruled out','2025-11-09 12:00:00'),
(11,1,'Normal','2025-10-02 09:00:00'),
(12,2,'Minor issues','2025-09-15 09:45:00'),
(13,3,'Abnormal','2025-08-21 05:00:00'),
(14,4,'Normal','2025-07-10 11:00:00'),
(15,5,'Recovering','2025-06-12 14:30:00'),
(16,1,'Normal','2025-05-22 10:00:00'),
(17,2,'Critical','2025-04-19 06:30:00'),
(18,3,'High','2025-03-19 16:00:00'),
(19,4,'Normal','2025-02-09 11:30:00'),
(20,5,'Improving','2025-01-18 13:30:00'),
(21,1,'Normal','2024-12-06 09:30:00'),
(22,1,'Abnormal','2024-11-28 15:00:00'),
(23,3,'Normal','2024-10-18 06:00:00'),
(24,4,'Normal','2024-09-04 11:30:00'),
(25,5,'Healing','2024-08-12 14:00:00'),
(26,1,'Normal','2024-07-06 10:30:00'),
(27,2,'Normal','2024-06-09 11:00:00'),
(28,3,'Normal','2024-05-18 13:00:00'),
(29,4,'Improving','2024-04-12 10:00:00'),
(30,5,'Critical','2024-03-16 06:00:00');
```

```
-- ===== 17. Billing (approx. 30) =====
```

```
INSERT INTO Billing (encounter_id, insurance_id, total_amount, B_status) VALUES
```

```
(1,1,650.00,'paid'),
```


(2,2,1720.00,'unpaid'),
(3,NULL,560.00,'paid'),
(4,NULL,300.00,'unpaid'),
(5,NULL,2000.00,'unpaid'),
(6,6,500.00,'paid'),
(7,NULL,600.00,'unpaid'),
(8,NULL,1800.00,'unpaid'),
(9,NULL,400.00,'unpaid'),
(10,NULL,2200.00,'unpaid'),
(11,NULL,350.00,'paid'),
(12,10,450.00,'unpaid'),
(13,NULL,700.00,'paid'),
(14,NULL,1200.00,'unpaid'),
(15,NULL,500.00,'unpaid'),
(16,NULL,300.00,'paid'),
(17,NULL,600.00,'unpaid'),
(18,NULL,1500.00,'unpaid'),
(19,NULL,400.00,'paid'),
(20,NULL,2000.00,'unpaid'),
(21,NULL,300.00,'paid'),
(22,NULL,450.00,'unpaid'),
(23,NULL,700.00,'paid'),
(24,NULL,1200.00,'unpaid'),
(25,NULL,500.00,'unpaid'),
(26,NULL,300.00,'paid'),
(27,NULL,600.00,'unpaid'),
(28,NULL,1500.00,'unpaid'),
(29,NULL,400.00,'paid'),
(30,NULL,2000.00,'unpaid');

-- ===== 18. Payments (approx. 25) =====

INSERT INTO Payment (billing_id, amount_paid, pay_mode, payment_date) VALUES
(1,650.00,'cash','2025-11-01 10:00:00'),
(3,560.00,'insurance','2025-11-04 06:00:00'),
(6,500.00,'cash','2025-11-06 11:00:00'),
(11,350.00,'cash','2025-10-02 12:00:00'),
(13,700.00,'insurance','2025-08-21 07:00:00'),
(16,300.00,'cash','2025-05-22 10:30:00'),

(19,400.00,'cash','2025-02-09 12:00:00'),
(21,300.00,'card','2024-12-06 09:45:00'),
(23,700.00,'insurance','2024-10-18 06:30:00'),
(26,300.00,'cash','2024-07-06 11:00:00'),
(29,400.00,'card','2024-04-12 10:15:00');